IVA — WhatsApp Channel: Fresh-Start Implementation Prompt

You are a senior Django engineer. You are starting **from an empty folder**. Any earlier IVA code has been deleted; do not look for it or reuse it.

Project root: use the current folder as the project root (the `iva-whatsapp/` folder in the tree). Do not create a subfolder. The only expected existing items are `CLAUDE.md`, `ARCHITECTURE.md` and the `Reference/` folder; keep them. If you find any other leftover files, list them and ask before deleting anything. Read `CLAUDE.md` and `ARCHITECTURE.md` before starting, and keep both up to date as stages complete.

---

0. Context and scope

IVA is a conversational food-ordering service on WhatsApp for the brand Ivar (healthy packaged foods, snacks, fresh bowls and drinks). Customers chat in English, Telugu or Hindi to browse products, build a cart, check out and track orders.

Our team builds only the WhatsApp channel:

1. WhatsApp webhook and Cloud API client
2. Conversation engine
3. Intent layer

The organization's backend team builds everything else: catalog, customers/profiles, cart, orders, payments, delivery, and the infrastructure (PostgreSQL, Redis, hosting). Their backend is a Django + Django REST Framework project. Our apps will later be copied into it.

This means:

- Our code talks to the backend **only through the Backend Port** (section 5). It never imports backend models or queries backend tables.
- Until their backend exists, we build and test against a **Fake Backend** that implements the same port.
- We create a **minimal host Django project** only so our apps can run and be tested. It is disposable: when we move to the organization's project, only our apps go.

---

1. Non-negotiable rules

1. **Django** (5.2 LTS). Our apps are Django apps. Business logic lives in each app's own modules (`services.py` and clearly named files), never in a top-level `services/` folder.
2. **Portability.** Everything we deliver lives in four packages under `apps/`: `whatsapp`, `conversations`, `ai`, and `iva_common` (shared helpers). These four must **not** import from `config/`, `host/`, or any other package. The only integration points are Django settings (read with defaults) and the Backend Port. A static test must enforce this.
3. **NO ADMIN PANEL of any kind.** No `django.contrib.admin`, no `/admin/` route, no `admin.py` files, no admin dashboard, frontend or UI. Tests must enforce this.
4. **Never fabricate** prices, availability, payment status, order status, delivery status, ETA or tracking. Every such value comes from the Backend Port. If the port returns nothing, tell the customer the information is unavailable.
5. **The LLM never writes to the database** and never decides price, stock, payment status, order status or availability. It only proposes a structured intent, which is schema-validated and then executed through the Backend Port.
6. **The Fake Backend is for development and tests only.** Production settings must refuse to start if it is configured.
7. **Secrets only in environment variables** / the git-ignored `.env`. Never print, log or commit tokens, app secrets, API keys or passwords. Commit `.env.example` with names only.
8. Customers never see tracebacks, exception text, internal IDs or service details. They get short, friendly messages in their language.
9. **Mask phone numbers and WhatsApp IDs** in all logs (e.g. `91******1234`).
10. **Redis** holds temporary conversation state. **PostgreSQL** holds persistent conversation records (messages, deduplication).
11. **Tests at every stage.** Do not move to the next stage while any test fails. Never claim a test passed unless you ran it; show real output.
12. **Scope:** Phase 1 food-ordering flow only. No Subscribe & Save, meal plans, Build a Box, AI Planner, Corporate Orders, Wellness, Yoga, marketing broadcasts or proactive notifications.
13. **Dependencies:** only those listed in this prompt. Prefer the standard library. Ask before adding anything else, and say why.

Infrastructure policy

- **PostgreSQL is provided by the organization.** Do not install it. A team member puts `DATABASE_URL` in `.env`. It must be a **development** database; if unsure, ask. The database user needs `CREATEDB` so tests can create and drop their own test database; if it lacks that, tell me exactly what is needed. If `.env` is missing or the connection fails, report the error type (never credentials), finish everything that doesn't need a database, and wait.
- **Redis:** if the organization provides `REDIS_URL`, `CELERY_BROKER_URL` and `CONVERSATION_REDIS_URL`, connect to them and do not install Redis. Otherwise a user-level install inside git-ignored `.local-services/`, bound to `127.0.0.1`, is allowed. **Ask first** before anything needing admin rights, UAC, a reboot, WSL, Docker, Windows features, system services, PATH/registry/firewall changes, or unofficial/third-party builds. When asking, list the options, what each changes, and your recommendation.
- Never stop or modify remote/shared services. Simulate failures in tests.
- Download only from official sources; verify checksums where published.
- The development machine is **Windows**. Document Windows commands (e.g. Celery needs `--pool=solo` there).

---

2. Stages

Stage	Content	Needs PostgreSQL	Needs Redis
S1 ← this session	Host project + Backend Port + Fake Backend	only for 2 host tests	no
S2	WhatsApp webhook + Cloud API client	yes	yes (Celery)
S3	Conversation engine	yes	yes
S4	Intent layer	no	no
S5	Hardening + end-to-end conversation tests	yes	yes

This session: implement S1 only, then stop and report.

---

3. Project layout (final target; S1 creates only what is marked)

```
iva-whatsapp/
├── manage.py                          [S1]
├── pytest.ini                         [S1]
├── conftest.py                        [S1]
├── ruff.toml                          [S1]
├── .env.example                       [S1]
├── .gitignore                         [S1]  # .env, .venv, .local-services/
├── README.md                          [S1]
├── requirements/
│   ├── base.txt                       [S1]
│   ├── dev.txt                        [S1]
│   └── prod.txt                       [S1]
├── config/                            [S1]  # HOST ONLY — not delivered
│   ├── __init__.py                    # loads Celery app
│   ├── celery.py
│   ├── urls.py
│   ├── asgi.py
│   ├── wsgi.py
│   └── settings/
│       ├── base.py
│       ├── dev.py
│       ├── test.py
│       └── prod.py
├── host/                              [S1]  # HOST ONLY — health checks, no-admin tests
│   ├── apps.py
│   ├── views.py
│   ├── urls.py
│   └── tests/
└── apps/                                    # DELIVERED — the portable part
    ├── __init__.py                    [S1]
    ├── iva_common/                    [S1]  # plain package, not a Django app
    │   ├── logging.py                 # phone/WA-ID masking filter
    │   ├── money.py                   # Decimal helpers, ₹ formatting
    │   └── tests/
    ├── conversations/                 [S1: port, fake, adapters, fixture]
    │   ├── apps.py
    │   ├── backend_port.py            [S1]
    │   ├── fake_backend.py            [S1]
    │   ├── adapters.py                [S1]
    │   ├── fixtures/ivar_catalog.json [S1]
    │   ├── models.py                  [S2/S3]
    │   ├── state.py                   [S3]
    │   ├── engine.py                  [S3]
    │   ├── messages/                  [S3]  # en.py, te.py, hi.py
    │   ├── renderers.py               [S3]
    │   └── tests/
    ├── whatsapp/                      [S2]
    │   ├── views.py  security.py  parser.py  client.py  tasks.py  urls.py
    │   └── tests/
    └── ai/                            [S4]
        ├── intents.py  rules.py  aliases.py  llm.py  validation.py  pipeline.py
        └── tests/
```

Allowed import direction inside the delivered part: `whatsapp → conversations → ai → iva_common`, and all of them may use `iva_common`. `ai` must not import `whatsapp`. Nothing imports `config` or `host`.

---

4. S1 part A — Host project

4.1 Dependencies (pin exact versions at install time)

- **base.txt:** Django 5.2 LTS, psycopg (v3, binary extra), django-environ, redis, celery
- **dev.txt:** `-r base.txt`, pytest, pytest-django, pytest-cov, ruff
- **prod.txt:** `-r base.txt`, gunicorn

DRF is not needed by our apps: the webhook must read the raw request body for signature verification, so it will be a plain Django view. (The organization's project has DRF; our apps don't depend on it.) Run `pip check` after installing.

4.2 Environment variables (S1)

```
DJANGO_SETTINGS_MODULE=config.settings.dev
DJANGO_SECRET_KEY=
DJANGO_DEBUG=false
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
DJANGO_CSRF_TRUSTED_ORIGINS=
DATABASE_URL=
DB_CONN_MAX_AGE=60
DB_CONNECT_TIMEOUT=5
REDIS_URL=
CELERY_BROKER_URL=
CONVERSATION_REDIS_URL=
IVA_BACKEND_ADAPTER=apps.conversations.fake_backend.FakeBackend
LOG_LEVEL=INFO
```

WhatsApp and LLM variables are added in S2 and S4, not now. If `.env` does not exist, create it from `.env.example` and generate a strong random `DJANGO_SECRET_KEY` yourself. Leave `DATABASE_URL` and Redis URLs for the team member unless you installed Redis locally under the policy.

4.3 Settings

base.py

- `INSTALLED_APPS`: `django.contrib.contenttypes`, `host`, `apps.conversations`. Nothing else. No admin, auth, sessions, messages or staticfiles.
- Middleware: security, common, clickjacking. (CSRF middleware is fine to include; the webhook view will be explicitly exempt in S2 because Meta authenticates by signature.)
- `DATABASES` from `DATABASE_URL`, `CONN_MAX_AGE` and `connect_timeout` from env, `CONN_HEALTH_CHECKS = True`, `ATOMIC_REQUESTS = False`.
- `CACHES` default: Django's built-in `RedisCache` at `REDIS_URL` if set, else `LocMemCache`.
- Logging: console, level from `LOG_LEVEL`, with `apps.iva_common.logging.MaskingFilter` attached to all handlers (also masks inside exception tracebacks).
- `TIME_ZONE = "Asia/Kolkata"`, `USE_TZ = True`.
- `IVA_BACKEND_ADAPTER` from env.
- Celery (namespace `CELERY_`): JSON only, `task_acks_late=True`, `worker_prefetch_multiplier=1`, soft/hard time limits, `broker_connection_retry_on_startup=True`, no result backend.
- JSON handlers for 404 and 500 (no HTML error pages, no details).

dev.py: `DEBUG=True`; if `DJANGO_SECRET_KEY` is empty, use a clearly labelled insecure dev-only key. **test.py:** eager Celery (`CELERY_TASK_ALWAYS_EAGER`, `CELERY_TASK_EAGER_PROPAGATES`), `LocMemCache`, real PostgreSQL (never SQLite), fallback insecure test key allowed. **prod.py:** `DEBUG=False`; HTTPS redirect (exempt only `^health/`), HSTS, secure cookies, `SECURE_PROXY_SSL_HEADER`, nosniff, `X_FRAME_OPTIONS="DENY"`; explicit hosts/origins, **no wildcards**. Raise `ImproperlyConfigured` if `DJANGO_SECRET_KEY`, `DATABASE_URL`, `REDIS_URL` or `CELERY_BROKER_URL` is missing, or if `IVA_BACKEND_ADAPTER` is unset or points to the Fake Backend. `wsgi.py` / `asgi.py` default to prod; `manage.py` defaults to dev.

4.4 Host health checks (`host/`)

- `GET /health/live/` → `200 {"status": "ok"}`, no dependency checks.
- `GET /health/ready/` → checks PostgreSQL (`SELECT 1`) and, if configured, Redis (`PING`). `200` when all OK, `503` otherwise, body `{"status": ..., "checks": {"database": "ok|fail", "redis": "ok|fail|not_configured"}}`. Never include error text, hostnames or versions; log failures server-side (masked).
- GET only. `config/urls.py` includes only `host.urls` in S1. No `/admin/`.

---

5. S1 part B — Backend Port contract

This contract is also shared with the organization's backend team. Implement it in `apps/conversations/backend_port.py` using `typing.Protocol` and frozen dataclasses. Money is always `Decimal` plus a currency code, never float.

5.1 Data types

```
CustomerRef       id: str, whatsapp_id: str, name: str|None, city: str|None,
                  language: str|None ("en"|"te"|"hi"),
                  diet: str|None ("veg"|"egg"|"non_veg"),
                  onboarding_complete: bool

Category          id: str, name: str, parent_id: str|None, sort_order: int

ProductSummary    id: str, name: str, category_id: str, diet_type: str,
                  price: Decimal, currency: str, in_stock: bool,
                  protein_g: Decimal|None, calories_kcal: int|None,
                  tags: tuple[str, ...], pack_size: str|None

Variant           id: str, label: str, price: Decimal, currency: str, in_stock: bool

ProductDetail     all ProductSummary fields + description: str|None,
                  variants: tuple[Variant, ...], nutrition_verified: bool

CartLine          line_id: str, product_id: str, variant_id: str, name: str,
                  quantity: int, unit_price: Decimal, line_total: Decimal

CartView          lines: tuple[CartLine, ...], subtotal: Decimal, currency: str

Address           id: str, label: str|None, text: str

CheckoutResult    order_number: str, order_status: str, payment_status: str,
                  total: Decimal, currency: str, payment_url: str|None

OrderStatusEvent  status: str, at: datetime

OrderStatusView   order_number: str, order_status: str, payment_status: str,
                  events: tuple[OrderStatusEvent, ...],
                  eta_start: datetime|None, eta_end: datetime|None,
                  tracking_id: str|None
```

Order statuses: `PENDING, CONFIRMED, PROCESSING, PACKED, OUT_FOR_DELIVERY, DELIVERED, CANCELLED`. Payment statuses: `PENDING, PAID, FAILED, REFUNDED`.

5.2 Operations

```
CustomerPort
  get_or_create_customer(whatsapp_id) -> CustomerRef
  update_profile(customer_id, *, name=None, city=None, language=None, diet=None,
                 onboarding_complete=None) -> CustomerRef

CatalogPort
  list_categories(parent_id=None) -> list[Category]
  list_products(*, category_id=None, search=None, diet=None, tag=None,
                limit=10, offset=0) -> tuple[list[ProductSummary], int]   # (page, total)
  get_product(product_id) -> ProductDetail
  resolve_product_names(text, limit=5) -> list[ProductSummary]

CartPort
  get_cart(customer_id) -> CartView
  add_item(customer_id, variant_id, quantity) -> CartView
  update_quantity(customer_id, line_id, quantity) -> CartView
  remove_item(customer_id, line_id) -> CartView

AddressPort
  list_addresses(customer_id) -> list[Address]
  add_address(customer_id, text, label=None) -> Address

CheckoutPort
  checkout(customer_id, address_id, idempotency_key) -> CheckoutResult

OrderPort
  get_latest_order(customer_id) -> OrderStatusView | None
  get_order(customer_id, order_number) -> OrderStatusView | None
  list_recent_orders(customer_id, limit=5) -> list[OrderStatusView]
```

Checkout safety (enforced by the backend, respected by us): the backend validates the cart, creates a PENDING order with PENDING payment and returns a payment URL. Only a verified provider confirmation makes payment PAID and then the order CONFIRMED. We never set or infer these statuses; we only display what the port returns. `idempotency_key` must be stable per checkout attempt so a retried WhatsApp message cannot create two orders.

5.3 Errors

```
BackendError (base)
├── BackendUnavailable   # timeout, connection failure, 5xx
├── NotFound             # product / order / cart line does not exist
├── ProductUnavailable   # out of stock or inactive
├── InvalidQuantity
├── CartEmpty
├── CheckoutRejected     # carries a safe reason code
└── NotAuthorized        # customer does not own the resource
```

5.4 Adapter loading (`adapters.py`)

- `get_backend()` imports the class at `settings.IVA_BACKEND_ADAPTER` (default: the Fake Backend in dev/test) and returns one shared instance.
- Raises a clear configuration error if the path is invalid or the class does not implement every port.

---

6. S1 part C — Fake Backend and catalog fixture

6.1 Fake Backend (`fake_backend.py`)

- In-memory and deterministic. No database tables. Loads `fixtures/ivar_catalog.json`.
- Implements every port operation and raises the correct typed errors.
- `resolve_product_names`: simple case-insensitive substring and token match on product names (the smarter multilingual matching belongs to the intent layer in S4).
- `checkout` creates a PENDING order with PENDING payment and a clearly fake URL `https://example.invalid/pay/<order_number>`. The same `idempotency_key` returns the same order. Order numbers look like `IVA1001`, `IVA1002`, ...
- **Test-only helpers** (clearly named, not part of the port): `simulate_payment_confirmed(order_number)` (payment → PAID, then order → CONFIRMED, each with an event), `simulate_payment_failed(order_number)`, `simulate_status(order_number, status)`, `set_stock(product_id, in_stock)`, `reset()`.
- Log each call with `backend=fake` so fake data is always identifiable.

6.2 Catalog fixture (`fixtures/ivar_catalog.json`)

Header: `"_note": "Development fixture. Prices and nutrition unverified. Not for production."`

Rules:

- Two top-level groups: `fresh` ("Fresh & Ready to Eat") and `packaged` ("Packaged Foods").
- `fresh` contains: Café Favourites, Protein Bowls, Non-Veg Snacks, Wellness Shots.
- `packaged` contains: Breakfast & Grains, Protein & Active Nutrition, Nuts Seeds & Snacks, Cooking Essentials, Teas & Beverages, Ready Mixes, Energy Bites & Cookies.
- One variant per product; `pack_size` only where listed; `in_stock: true`; `nutrition_verified: false`; `currency: "INR"`.
- Tags: `high_protein` if protein ≥ 20 g; `light` if calories ≤ 250 kcal.
- Blank protein = `null`.

Category	Product	₹	Protein g	kcal	Diet	Pack
Breakfast & Grains	Ivar Oats	249	6	180	veg	
Breakfast & Grains	Multigrain Breakfast Mix	329	7	200	veg	
Breakfast & Grains	Ragi Mix	299	5	190	veg	
Breakfast & Grains	Millet Mix	289	6	185	veg	
Protein & Active Nutrition	Plant Protein Mix	899	24	130	veg	
Protein & Active Nutrition	Active Protein	949	25	140	veg	
Protein & Active Nutrition	Ivar Active Nutrition Drinks	79		40	veg	
Nuts, Seeds & Snacks	Seed Mix	349	6	160	veg	
Nuts, Seeds & Snacks	Roasted Nuts	399	6	170	veg	
Nuts, Seeds & Snacks	Trail Mix	349	5	165	veg	
Nuts, Seeds & Snacks	Makhana Crunch	199	4	120	veg	
Nuts, Seeds & Snacks	Roasted Chana Mix	179	7	140	veg	
Nuts, Seeds & Snacks	Ragi Crisps	189	3	145	veg	
Nuts, Seeds & Snacks	Millet Puffs	179	3	110	veg	
Nuts, Seeds & Snacks	Millet & Seed Crackers	219	4	150	veg	
Nuts, Seeds & Snacks	Jowar Crunch	179	3	140	veg	
Nuts, Seeds & Snacks	Jowar Puffs	179	3	115	veg	
Cooking Essentials	Multigrain Atta	249	12	340	veg	1 kg
Cooking Essentials	Quinoa	449	13	360	veg	500 g

Cooking Essentials	Chia Seeds	299	5	140	veg	
Cooking Essentials	Flax Seeds	199	5	150	veg	200 g
Teas & Beverages	Herbal Tea	299		5	veg	
Teas & Beverages	Green Tea	279		2	veg	
Teas & Beverages	Turmeric Latte	329	2	90	veg	
Ready Mixes	Instant Porridge Mix	279	5	180	veg	
Ready Mixes	Pancake Mix	299	6	210	veg	
Ready Mixes	Smoothie Mix	349	4	170	veg	
Café Favourites	Classic Oats Bowl	199	7	220	veg	
Café Favourites	Ivar Muesli Bowl	219	7	240	veg	
Café Favourites	Millet Pancakes	249	6	230	veg	
Café Favourites	Ragi Upma	179	5	210	veg	
Café Favourites	Millet Dosa	149	4	180	veg	
Café Favourites	Protein Smoothie	229	18	220	veg	
Café Favourites	Fruits, Nuts & Seeds Bowl	249	7	260	veg	
Non-Veg Snacks	Chicken Pepper Bites	279	22	210	non_veg	
Non-Veg Snacks	Fish Crispy Bites	299	20	200	non_veg	
Non-Veg Snacks	Prawn Masala Bites	329	21	190	non_veg	
Non-Veg Snacks	Egg Masala Crunch	229	14	180	egg	
Non-Veg Snacks	Chicken Seekh Kebab Bites	299	23	220	non_veg	
Energy Bites & Cookies	Date & Chocolate Protein Bites	249	8	140	veg	
Energy Bites & Cookies	Mango Coconut Fruit Bites	229	2	120	veg	
Energy Bites & Cookies	Millet Chocolate Bites	239	4	130	veg	
Energy Bites & Cookies	Oats & Millet Cookies	219	4	150	veg	
Energy Bites & Cookies	Peanut Sesame Jaggery Bites	199	6	160	veg	
Energy Bites & Cookies	Ragi Cocoa Cookies	219	4	145	veg	
Wellness Shots	Morning Boost Shot	99		25	veg	30 ml
Wellness Shots	Night Reset Shot	99		20	veg	30 ml
Wellness Shots	Immunity Shield Shot	129		20	veg	30 ml
Wellness Shots	Digest Ease Shot	99		15	veg	30 ml
Wellness Shots	Breath Fresh Shot	99		15	veg	30 ml
Protein Bowls	Garlic Steak & Prawns Bowl	449	38	420	non_veg	
Protein Bowls	Grilled Chicken Quinoa Bowl	379	35	400	non_veg	
Protein Bowls	Lemon Herb Salmon Bowl	469	32	380	non_veg	
Protein Bowls	High-Protein Paneer Bowl	329	26	360	veg	
Protein Bowls	Egg & Avocado Power Bowl	299	24	340	egg	

7. S1 tests (all must pass)

Host

1. `django.contrib.admin` is not installed; `GET /admin/` returns 404; no `admin.py` exists anywhere in the repo.
2. `/health/live/` returns 200.
3. `/health/ready/` returns 200 with a working database *(needs PostgreSQL)*.
4. `/health/ready/` returns 503 when the database check fails (mocked) and when Redis fails (mocked), with no error text in the body.
5. Prod settings raise `ImproperlyConfigured` when the secret key is missing, when a host/origin contains a wildcard, and when the adapter is unset or is the Fake Backend. Prod has `DEBUG=False`.
6. HTTPS redirect in prod exempts only `/health/...` paths.
7. `makemigrations --check --dry-run` reports no changes *(needs PostgreSQL)*.

Delivered apps

1. Portability: a static check proves `apps/whatsapp`, `apps/conversations`, `apps/ai`, `apps/iva_common` import nothing from `config` or `host`, and follow the allowed import direction.
2. Masking filter masks phone numbers and WhatsApp IDs in messages, args and tracebacks.
3. Fixture: exactly 55 products, 11 categories, 2 groups, with the per-category counts in the table; tags follow the rules exactly; Egg Masala Crunch and Egg & Avocado Power Bowl are `egg`.
4. Every port operation works on the Fake Backend (happy path), and every typed error is raised in the right case (unknown product, out of stock, invalid quantity, empty-cart checkout, another customer's cart line or order).
5. Money is `Decimal` everywhere; cart subtotal equals the sum of line totals.

6. Checkout returns PENDING/PENDING; the same idempotency key returns the same order; `simulate_payment_confirmed` gives PAID then CONFIRMED with events; `simulate_payment_failed` gives FAILED and the order stays PENDING.
7. `get_latest_order` returns `None` for a customer with no orders.
8. **Contract test suite:** parametrized over the adapter class so the backend team can later run the same tests against their real adapter. Keep it in `apps/conversations/tests/contract/`.
9. `get_backend()` loads the configured adapter and fails clearly for an invalid path or incomplete class.
10. `pip check`, `ruff check .` and `ruff format --check .` pass.

If PostgreSQL is not available yet, run everything else (`pytest -v -m "not django_db"`), list exactly which tests are pending and why, and do not mark S1 complete.

---

8. Later stages (context only — do not implement now)

S2 — Webhook and Cloud API client

- `GET /webhooks/whatsapp/`: Meta verification; compare `hub.verify_token` in constant time; echo `hub.challenge`.
- `POST /webhooks/whatsapp/`: CSRF-exempt plain Django view; verify `X-Hub-Signature-256` (HMAC-SHA256 over the **raw body** with the app secret, constant-time compare); invalid → 403, nothing processed. Parse; dedupe by WhatsApp message ID with a database unique constraint; enqueue a Celery task; return 200 fast. Status callbacks are recorded, never replied to.
- Client: text, reply buttons (max 3), list messages (max 10 rows total), text length limits, timeouts, retry with backoff on 429/5xx, no token logging. Reply only to user-initiated conversations.
- Env: `WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_APP_SECRET`, `WHATSAPP_VERIFY_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_API_VERSION`, `WHATSAPP_GRAPH_BASE_URL`.

S3 — Conversation engine

- States: NEW → LANGUAGE → ONBOARD_NAME → ONBOARD_CITY → ONBOARD_DIET (optional, skippable, changeable later) → MAIN_MENU → GROUP → CATEGORY → PRODUCT → QUANTITY → CART → CHECKOUT_ADDRESS → CHECKOUT_CONFIRM → AWAITING_PAYMENT.
- Global intents from any state: order status, payment status, view cart, help, change language, menu.
- Returning customers skip onboarding. Menus come from the Backend Port, never hardcoded. Paginate lists beyond 10 rows. Diet preference filters product lists when set.
- Redis session store (db from `CONVERSATION_REDIS_URL`) with TTL; expired sessions resume at the main menu.
- Text in `messages/en.py`, `te.py`, `hi.py`; Telugu and Hindi flagged for native-speaker review.
- Env: `CONVERSATION_SESSION_TTL_SECONDS`.

S4 — Intent layer

- Pipeline: Button/List handler → Rule/Alias/Fuzzy matching → LLM fallback → Structured intent → Schema validation → Backend Port.
- Intents: GREETING, LANGUAGE_SELECT, PRODUCT_SEARCH, CATEGORY_SELECT, PRODUCT_SELECT, ADD_TO_CART, REMOVE_FROM_CART, UPDATE_QUANTITY, VIEW_CART, CHECKOUT, ORDER_STATUS, PAYMENT_STATUS, TRACKING, HELP, FALLBACK.
- Multilingual aliases (English, Telugu, Hindi, romanized Telugu/Hindi, mixed, e.g. "naaku snacks kavali", "mujhe protein chahiye"). Standard-library fuzzy matching first.
- LLM behind a provider-agnostic interface: timeout, strict JSON output, schema validation, confidence threshold → clarifying question; on LLM failure, fall back to rules without breaking the flow.
- Env: `LLM_PROVIDER`, `LLM_API_KEY`, `LLM_MODEL`, `LLM_TIMEOUT_SECONDS`.

S5 — Hardening

- End-to-end transcript tests for new user, returning user, discovery, cart, checkout, order status and payment status in English, Telugu, Hindi, Telugu-English, Hindi-English and romanized Telugu, against the Fake Backend.
- Failure tests: backend down, LLM down, WhatsApp API down, duplicate webhooks, invalid signatures, expired sessions.

---

9. Deliverables for this session

1. The file tree you created.
2. Pinned `requirements/*.txt` contents.
3. What you installed, if anything (name, version, source, location).
4. Setup and run commands for Windows (venv, install, `.env`, migrate, runserver, tests).
5. The **actual** output of `pip check`, `pytest -v` (state skipped/deselected tests and why), `ruff check .` and `ruff format --check .`.
6. Deviations from this prompt with reasons, or "No deviations."

7. Then **stop**. Do not start S2.